Ex7: Model Planning and Building – Clustering (K-Means)

Dataset: Mall Customer Segmentation Data (Kaggle) – `Mall_Customers.csv`

Upload `Mall_Customers.csv` to the Colab **Files** panel before running. If the file is not uploaded, the notebook downloads it from Kaggle automatically.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
```

Task 1: Load the Mall Customer Segmentation dataset into a Pandas DataFrame

```
file_name = "Mall_Customers.csv"
if not os.path.exists(file_name):  # file not uploaded -> download from Kaggle
    import kagglehub
    path = kagglehub.dataset_download("vjchoudhary7/customer-segmentation-tutorial-in-python")
    file_name = os.path.join(path, "Mall_Customers.csv")

df = pd.read_csv(file_name)
print("Dataset loaded:", df.shape)
```

```
Dataset loaded: (200, 5)
```

Task 2: Explore the dataset

First and last five records

```
print("First five records:")
display(df.head())
print("Last five records:")
display(df.tail())
```

```
First five records:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
Last five records:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

Shape and data types

```
print("Shape of the dataset:", df.shape)
print("\nData types:")
print(df.dtypes)
```

```
Shape of the dataset: (200, 5)

Data types:
CustomerID                int64
Gender                      str
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object
```

Missing values and duplicate records

```
print("Missing values in each column:")
print(df.isnull().sum())
print("\nNumber of duplicate records:", df.duplicated().sum())
```

```
Missing values in each column:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Number of duplicate records: 0
```

Summary statistics

```
display(df.describe().T)
```

	count	mean	std	min	25%	50%	75%	max
CustomerID	200.0	100.50	57.879185	1.0	50.75	100.5	150.25	200.0
Age	200.0	38.85	13.969007	18.0	28.75	36.0	49.00	70.0
Annual Income (k$)	200.0	60.56	26.264721	15.0	41.50	61.5	78.00	137.0
Spending Score (1-100)	200.0	50.20	25.823522	1.0	34.75	50.0	73.00	99.0

Task 3: Select the features for clustering – Annual Income (k$), Spending Score (1–100)

```
features = ['Annual Income (k$)', 'Spending Score (1-100)']
X = df[features]
display(X.head())
```

	Annual Income (k$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

Task 4: Standardise the selected features using StandardScaler

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

scaled_df = pd.DataFrame(X_scaled, columns=features)
print("Scaled features (mean ≈ 0, std ≈ 1):")
display(scaled_df.describe().round(3))
```

```
Scaled features (mean ≈ 0, std ≈ 1):
```

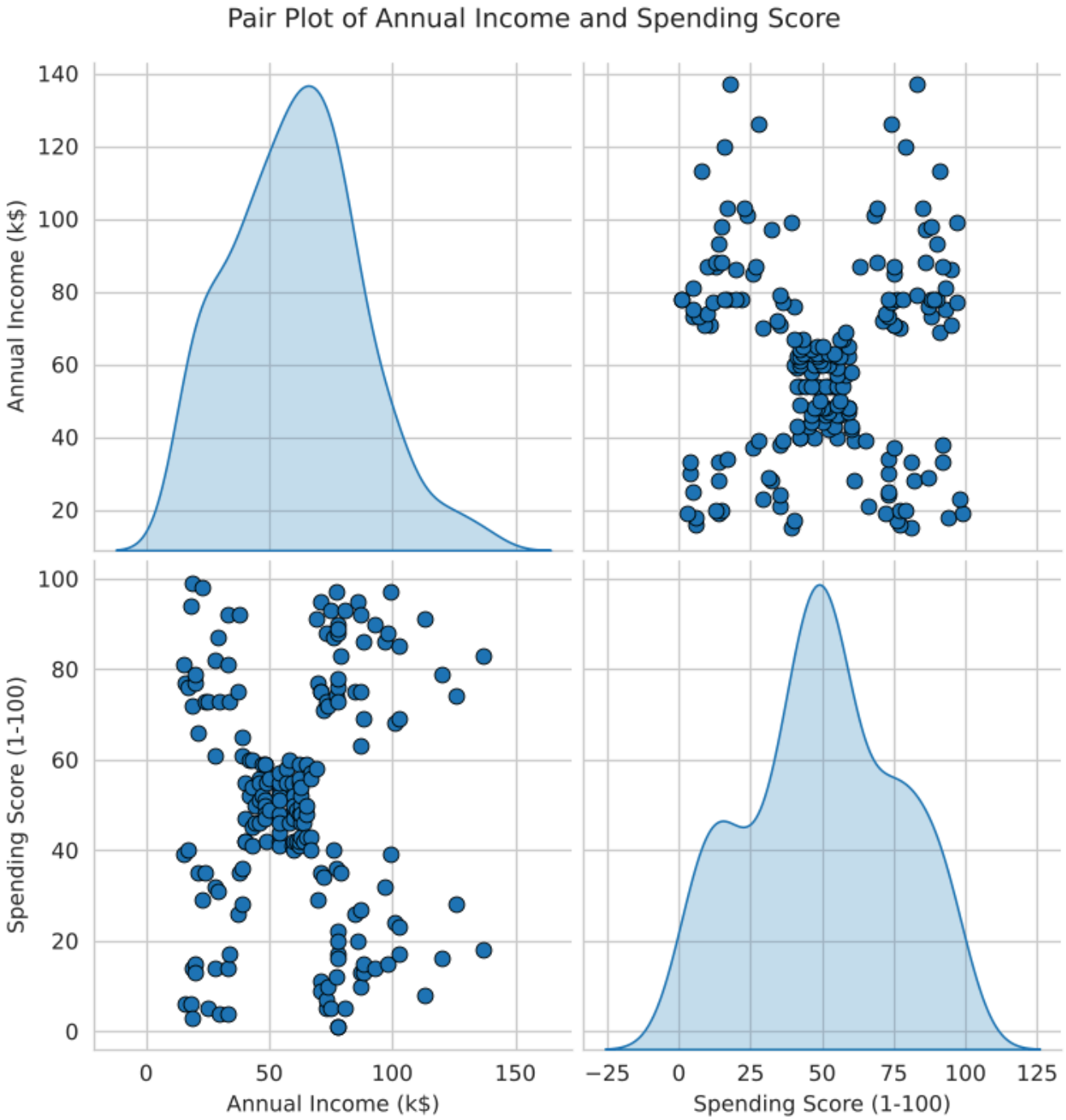
	Annual Income (k$)	Spending Score (1-100)
count	200.000	200.000
mean	-0.000	-0.000
std	1.003	1.003
min	-1.739	-1.910
25%	-0.728	-0.600
50%	0.036	-0.008
75%	0.666	0.885
max	2.918	1.894

Task 5: Visualise the selected features – Scatter Plot and Pair Plot

```
plt.figure(figsize=(9, 6))
sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)', s=70, color='steelblue', edgecolor='black')
plt.title('Annual Income vs Spending Score')
plt.show()
```




```
sns.pairplot(df[features], diag_kind='kde', height=3.5, plot_kws={'s': 50, 'edgecolor': 'black'})
plt.suptitle('Pair Plot of Annual Income and Spending Score', y=1.03)
plt.show()
```



Model Planning and Building

Task 6: Elbow Method – find the optimal number of clusters

```
k_values = range(1, 11)
wcss = []
for k in k_values:
    km = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    km.fit(X_scaled)
    wcss.append(km.inertia_)

display(pd.DataFrame({'K': list(k_values), 'WCSS (Inertia)': np.round(wcss, 2)}).set_index('K').T)

# Elbow point = the K whose point is farthest from the straight line joining the first and last points of the curve
points = np.column_stack([list(k_values), wcss])
line = points[-1] - points[0]
line = line / np.linalg.norm(line)
vectors = points - points[0]
distances = np.abs(vectors[:, 0] * line[1] - vectors[:, 1] * line[0])
optimal_k = list(k_values)[int(np.argmax(distances))]

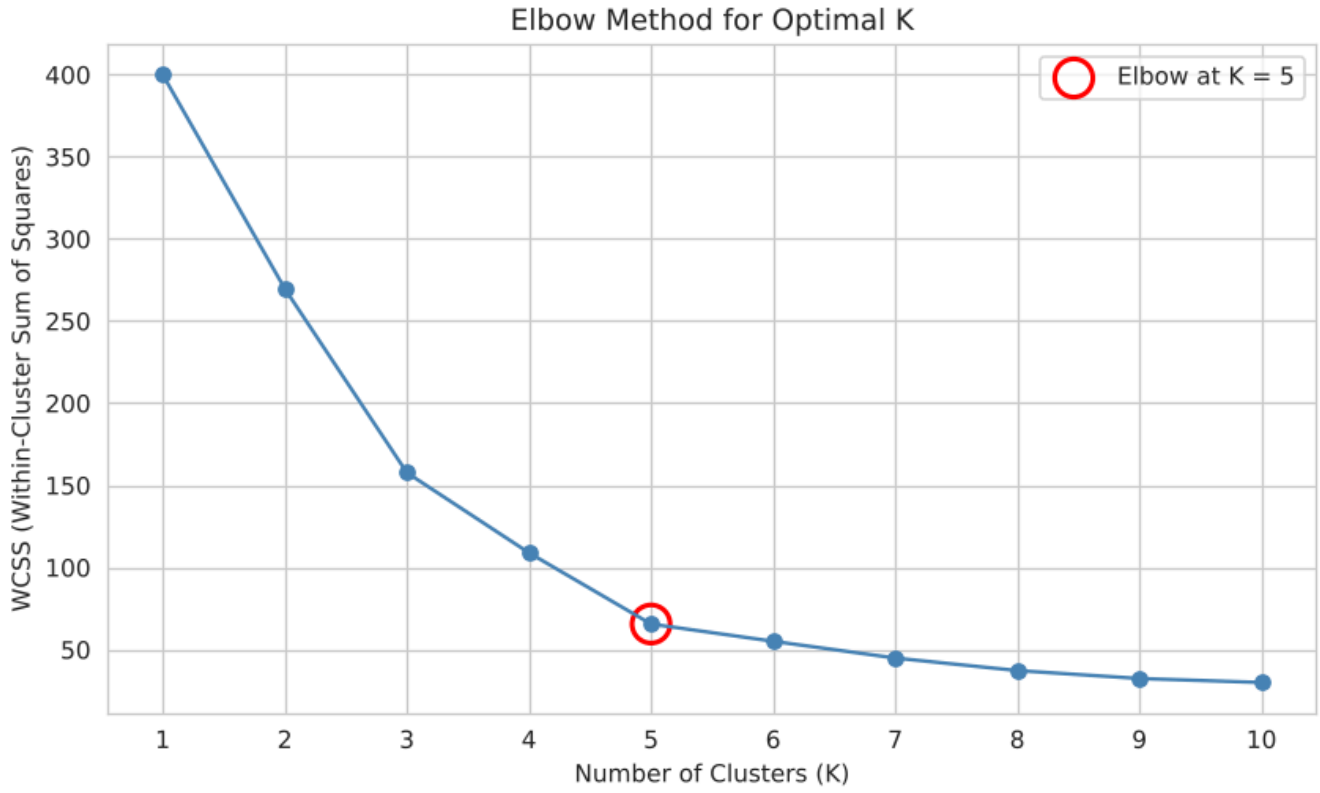
plt.figure(figsize=(9, 5))
plt.plot(list(k_values), wcss, marker='o', color='steelblue')
```

```
plt.scatter(optimal_k, wcss[optimal_k - 1], s=250, facecolors='none', edgecolors='red', linewidths=2,
            label=f'Elbow at K = {optimal_k}')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS (Within-Cluster Sum of Squares)')
plt.xticks(list(k_values))
plt.legend()
plt.show()

print("Optimal number of clusters (K):", optimal_k)
```

K	1	2	3	4	5	6	7	8	9
WCSS (Inertia)	400.0	269.69	157.7	108.92	65.57	55.06	44.86	37.23	32.39

K	10
WCSS (Inertia)	29.98




```
Optimal number of clusters (K): 5
```

Task 7: Train K-Means with the optimal K, add cluster labels to the dataset and save it

```
kmeans = KMeans(n_clusters=optimal_k, init='k-means++', n_init=10, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)

display(df.head(10))

df.to_csv('Mall_Customers_Clustered.csv', index=False)
print("Saved the clustered dataset as Mall_Customers_Clustered.csv")
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4
5	6	Female	22	17	76	2
6	7	Female	35	18	6	4
7	8	Female	23	18	94	2
8	9	Male	64	19	3	4
9	10	Female	30	19	72	2

```
Saved the clustered dataset as Mall_Customers_Clustered.csv
```

Cluster Visualisation

Task 8: Scatter plot of the clusters with highlighted centroids

```
centroids = scaler.inverse_transform(kmeans.cluster_centers_)   # back to original units
centroid_df = pd.DataFrame(centroids, columns=features).round(2)
centroid_df.index.name = 'Cluster'
print("Cluster centroid coordinates (original scale):")
display(centroid_df)

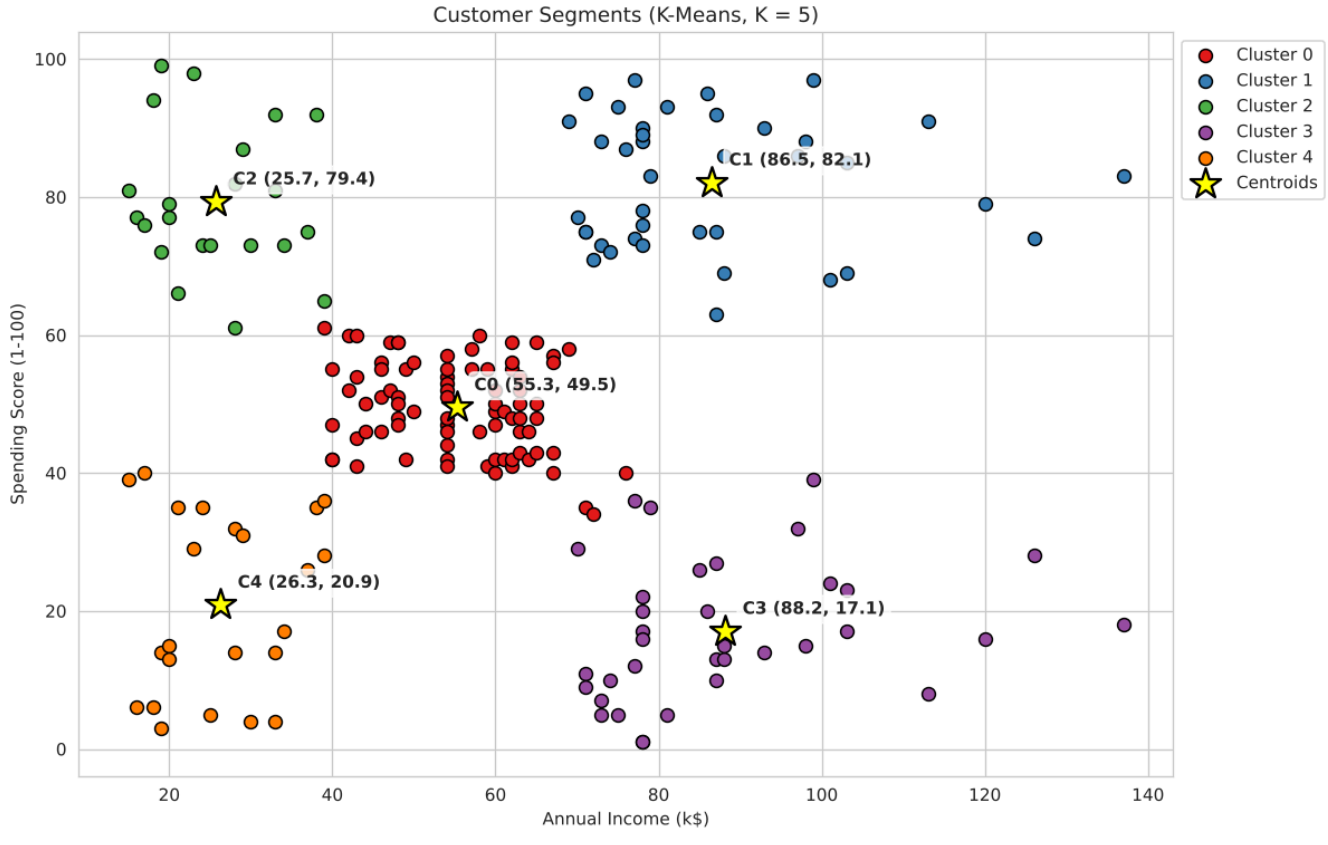
plt.figure(figsize=(11, 7))
palette = sns.color_palette('Set1', optimal_k)
for c in range(optimal_k):
    members = df[df['Cluster'] == c]
    plt.scatter(members['Annual Income (k$)'], members['Spending Score (1-100)'], s=60,
                color=palette[c], edgecolor='black', label=f'Cluster {c}')

plt.scatter(centroids[:, 0], centroids[:, 1], s=350, c='yellow', marker='*', edgecolor='black',
            linewidths=1.5, label='Centroids')
for c, (x, y) in enumerate(centroids):
    plt.annotate(f'C{c} ({x:.1f}, {y:.1f})', (x, y), textcoords='offset points', xytext=(10, 10),
                 fontsize=10, fontweight='bold',
                 bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.8))

plt.title(f'Customer Segments (K-Means, K = {optimal_k})')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
plt.tight_layout()
plt.show()
```

Cluster centroid coordinates (original scale):

Cluster	Annual Income (k$)	Spending Score (1-100)
0	55.30	49.52
1	86.54	82.13
2	25.73	79.36
3	88.20	17.11
4	26.30	20.91



Cluster Analysis

Task 9: Number of customers in each cluster

```
cluster_counts = df['Cluster'].value_counts().sort_index()
display(cluster_counts.to_frame('Number of Customers'))

plt.figure(figsize=(8, 5))
ax = sns.barplot(x=cluster_counts.index.astype(str), y=cluster_counts.values, hue=cluster_counts.index.astype(str),
                 palette=palette, legend=False)
for container in ax.containers:
    ax.bar_label(container)
plt.title('Number of Customers per Cluster')
plt.xlabel('Cluster')
plt.ylabel('Number of Customers')
plt.show()
```

Cluster	Number of Customers
0	81
1	39
2	22
3	35
4	23



Task 10: Average Annual Income and Spending Score for each cluster

```
cluster_profile = df.groupby('Cluster')[features].mean().round(2)
cluster_profile['Number of Customers'] = cluster_counts

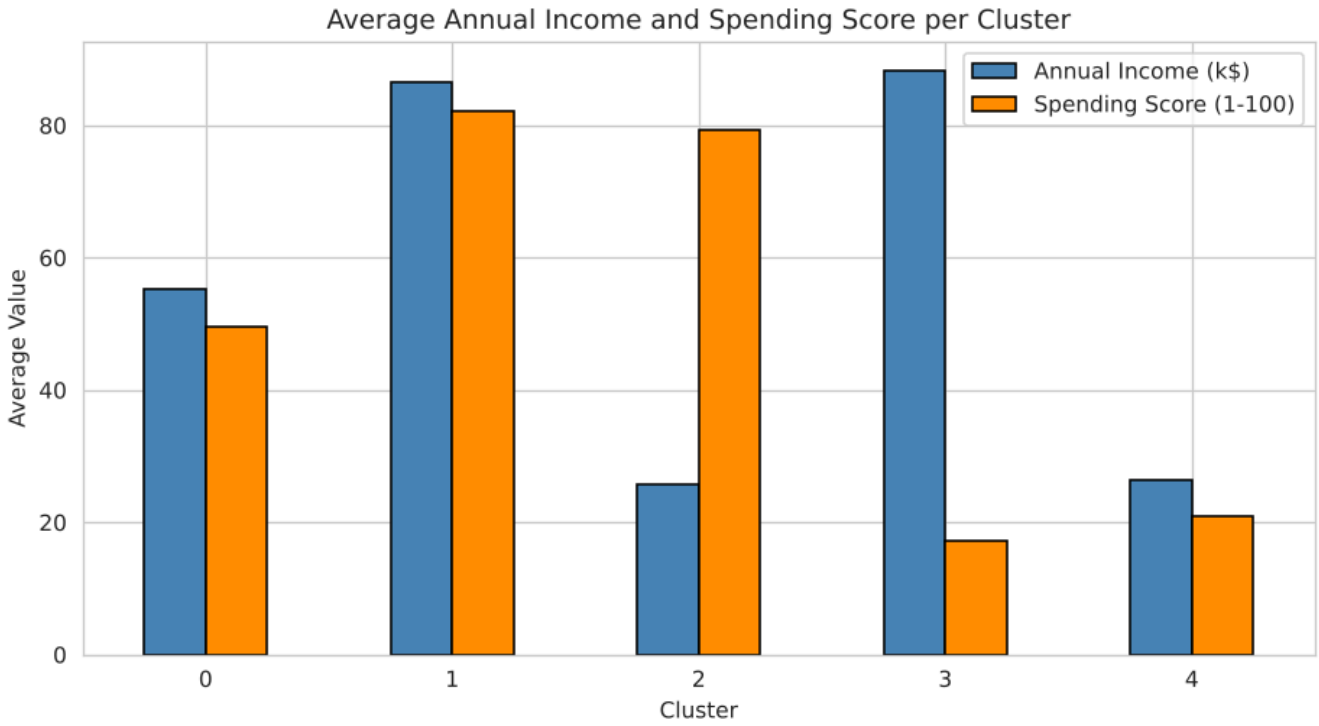
# Name each segment from its average income (low < 40k$, high > 70k$) and spending score (low < 35, high > 65)
def segment_name(row):
    income = row['Annual Income (k$)']
    score = row['Spending Score (1-100)']
    inc = 'High Income' if income > 70 else ('Low Income' if income < 40 else 'Average Income')
    spd = 'High Spending' if score > 65 else ('Low Spending' if score < 35 else 'Average Spending')
    return f'{inc} — {spd}'

cluster_profile['Segment'] = cluster_profile.apply(segment_name, axis=1)
display(cluster_profile)

cluster_profile[features].plot(kind='bar', figsize=(10, 5), color=['steelblue', 'darkorange'], edgecolor='black')
```

```
plt.title('Average Annual Income and Spending Score per Cluster')
plt.xlabel('Cluster')
plt.ylabel('Average Value')
plt.xticks(rotation=0)
plt.show()
```

Cluster	Annual Income (k$)	Spending Score (1-100)	Number of Customers	Segment
0	55.30	49.52	81	Average Income – Average Spending
1	86.54	82.13	39	High Income – High Spending
2	25.73	79.36	22	Low Income – High Spending
3	88.20	17.11	35	High Income – Low Spending
4	26.30	20.91	23	Low Income – Low Spending



Marketing strategies for each segment (match the segment names in the table above)

Segment	Strategy
High Income – High Spending	Premium / VIP customers: loyalty rewards, early access to new collections, personalised premium offers.
High Income – Low Spending	Can spend but don't: targeted campaigns showing quality and value, exclusive experiences, personal shopping to win them over.
Average Income – Average Spending	Largest, stable group: regular promotions, bundle deals and loyalty points to keep them engaged.
Low Income – High Spending	Enthusiastic but price-sensitive: discounts, flash sales, buy-now-pay-later and trendy affordable products.
Low Income – Low Spending	Budget customers: value packs, clearance sales and essential-item discounts.